Vector Space Search

Part A - Term Weighting [*60 marks*]

In class, we covered the concept of term weighting in a term by document matrix for vector space search. Specifically, we covered Term Frequency-Inverse Document Frequency and saw that it immediately had a positive effect for the search that was demonstrated in class.

There are many other term weighting schemes that can be employed. Another popular scheme is Log-Entropy as shown in the formula below:

$$A_{ij} = \log(1 + f_{ij}) \left[1 + \left(\sum_j \frac{p_{ij} \log(p_{ij})}{\log(n)} \right) \right]$$

where:

A_{ij} = Term × Document matrix weighted entry for term i in document j

f_{ij} = The term frequency of term i in document j

$p_{ij} = \frac{f_{ij}}{\sum_j f_{ij}}$
Proportion of term i in document j with respect to the total frequency count of f_i (term i)

n Total number of documents

NOTE, for the programming component you are given the same files and data that were demonstrated in class. These are:

- `inverted_index.py` - This will be the file you need to modify for the programming components. This contains the inverted index class/data structure as well as a simple vector space model search engine.
- `linked_list.py` - Simple linked list class. You won't need to change this.
- `utils.py` - Some generic file type utilities. You won't need to change this.
- `shakespeare` - Folder containing Shakespeare's works as demonstrated in class.

The Jupyter notebook used for the demo will also be included. Feel free to just modify that to try out/test your modifications and changes. Do use a jupyter notebook to demo your work.

[5 marks] (a) Given the formula for log entropy describe what the purpose of the first component is (the local weight logarithm).

$$\log(1 + f_{ij})$$

[5 marks] (b) Given the formula for Log-Entropy describe what the purpose of the second component is (the entropy calculation).

$$\left[1 + \left(\sum_j \frac{p_{ij} \log(p_{ij})}{\log(n)}\right)\right]$$

[30 marks] (c) The code and Jupyter Notebook demonstrated in class has been made available to you. You are to enhance this code further by implementing the Log-Entropy weighting scheme into the code. **NOTE:** for consistency please use the `math.log` function for logarithms you may calculate (natural logarithm).

There are two methods in the `inverted_index.py` source file you will need to enhance.

- `calcLogEntropy()` - this method should calculate the Log-Entropy value(s) for each term contained in the inverted index. These values are what will populate the term by document matrix as generated by the `generate_term_by_doc_matrix()` method. Note (as demonstrated in class) when you generate this matrix you need to ensure the Log-Entropy values are calculated already (as it pulls them from the inverted_index itself). [20 marks]
- `create_query_vector()` - This generates a query vector to be used in a search. Typically the query vector should be processed the same way as the data in the term by document matrix we will be applying the search to. So if we have generated it using only TF then a query vector should only contain TF values, same for TF-IDF or Log-Entropy. [10 marks]

Search for "`TODO`" in the code to find where this needs to be inserted.

Also make sure you understand the structure of the linked list and how the Log-Entropy values should be stored in the linked list. The `terms` list in the inverted index is a list of lists. Each list item has three values, `[x, y, z]` where:

- x - Term
- y - Linked list containing term x's posting list
- z - The total term count overall for term x over all the documents in y (this will be useful for your Log-Entropy calculations).

You can use the `binary_search_for_term(t)` method to lookup a term in the inverted index (this is also why we need to keep the `terms` list sorted). This returns two items, the list outlined above as well as the index of the term in the `terms` list itself[1]. The linked list items for each term, say t's, posting list contains a list of 4, `[ a, b, c, d ]` , values where:

- a - document identifier (document name basically)
- b - Term frequency for term this linked list (posting list) refers to in document *a* above.
- c - The TFIDF value for this term in this document if calculated (`None` otherwise.

[1] If you want to see the difference between a binary search and linear search, use the `search_for_term(t)` method instead. Be patient.

- d - The Log-Entropy value for this term in this document if calculated (`None` otherwise.

[20 marks] (d) As we did in class (but this time using Log-Entropy) perform the query "`scotland kings and thanes`" using the Log-Entropy weighting scheme. Make sure both the term by document matrix (A) is built using this as well as the query. Show the top 10 results [10 marks]. Does this weighting scheme work better or worse than TF only or TF-IDF and why or why not [10 marks]?

Part B - Comparison Metrics [*40 marks*]

The code given uses the cosine operator between two vectors to determine the similarity between two document or a document and a search query. As we are using vectors, many other comparison operators are possible. Some will work better than others.

[40 marks] (a) Implement four (4) comparison operators and modify the search method (or write a different one) in `inverted_index.py` for vectors and determine how well they work (or don't!) on our previous search. Implement and generate the top 10 results with the original query on the Shakespeare dataset for TF, TF-IDF and Log-Entropy [4 marks implementation, 6 marks results X 4]:

1. Euclidian distance (think about how you would rank this search).
2. Pearson correlation. The Python `numpy` library has this function (`corrcoef`).
3. Spearman correlation. The Python `scipy` library has this function (`stats.spearmanr`).
4. Comparison operator of your choice. Do a bit of a research to see what other vector comparison operators are out there. Is there anything that works well?